

NSS COLLEGE APPLICATION

Software Architecture & Technical Design Report

Project	National Service Scheme (NSS) Management Application
Document	System Architecture Report
Scope	Volunteers, events, attendance, service hours, reports, communication and administration
Architecture	Secure modular monolith with REST API and relational database

Executive Summary

This report defines a production-oriented architecture for a college NSS management application. The system centralizes volunteer records, NSS units, activities, event registration, attendance, service-hour calculation, announcements, documents, reports, achievements and administrative controls. The recommended design is a modular monolith: one deployable backend with strong domain boundaries, a relational database, object storage for files, and a web/mobile-friendly client. This provides lower operational complexity than microservices while preserving a clean path to scale.

Core architectural principle: keep business rules on the server, treat attendance and service hours as authoritative domain data, enforce role-based access control at every protected operation, and maintain an audit trail for sensitive changes.

1. System Overview

The application is intended to replace fragmented NSS workflows such as spreadsheets, paper attendance, messaging groups and manually maintained activity records. It should provide a single source of truth for volunteer participation and administrative reporting.

Primary actors

- Volunteer: maintains profile, views events, registers for eligible activities, checks attendance status, sees service history and receives notices.
- Student Leader: assists with event coordination and operational tasks permitted by the coordinator.
- Programme Officer / Faculty Coordinator: manages events, attendance, activities, reports, volunteers and approvals within assigned NSS units.
- Administrator: manages college-wide configuration, users, roles, units and audit/security controls.

Primary goals

- Centralize NSS volunteer and event data.
- Reduce attendance manipulation and duplicate/manual entry.
- Automatically calculate service participation from verified records.
- Generate reliable administrative reports.
- Provide secure access based on least privilege.
- Keep the system maintainable for future mobile applications or multiple units.

Non-goals for the first release

- Microservice decomposition.
- Complex AI decision-making for official attendance or eligibility.
- Replacing official college identity systems unless an integration is formally approved.

2. Functional Architecture

The system is divided into bounded modules. Modules should communicate through explicit application services and domain events rather than direct access to unrelated tables.

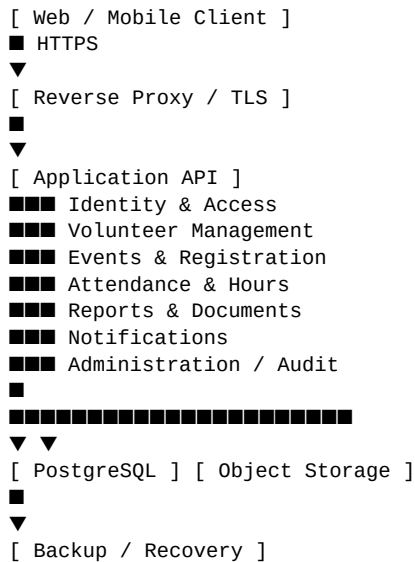
Module	Responsibilities
Identity & Access	Authentication, sessions/tokens, roles, permissions, account status, password/security policies.
Volunteer Management	Volunteer profile, NSS unit membership, enrollment status, verification and history.
Unit Management	NSS units, coordinators, student leaders, unit-level boundaries.
Event Management	Event lifecycle, schedules, venue, capacity, eligibility, registration and cancellation.
Attendance	Attendance sessions, check-in, coordinator verification, corrections and immutable audit records.
Service Hours	Derived participation totals based on verified attendance and approved activity rules.

Module	Responsibilities
Activity & Reporting	Post-event reports, objectives, outcomes, participant counts, attachments and approvals.
Communication	Announcements, notifications, reminders and delivery status.
Documents & Media	Controlled upload/download of photographs, PDFs and reports using object storage.
Achievements	Awards, leadership roles, recognitions and approved certificates.
Analytics & Export	Dashboards, filters, CSV/PDF exports and administrative statistics.
Audit & Administration	Audit events, configuration, privileged actions, data retention and system controls.

3. High-Level System Architecture

The recommended deployment is a client-server architecture. The client handles presentation and interaction; the backend owns authentication, authorization, validation and domain rules; PostgreSQL stores transactional data; object storage holds files; a background worker handles non-critical asynchronous jobs.

Logical architecture



Recommended technology stack

Layer	Recommendation	Reason
Frontend	React + TypeScript	Strong typing, component ecosystem, easy web deployment.
Backend	Node.js + TypeScript + NestJS	Modular structure, dependency injection, validation and maintainability.
Database	PostgreSQL	ACID transactions, constraints, indexing and mature relational tooling.
ORM	Prisma	Typed database access and migrations.
Object Storage	S3-compatible storage	Scalable storage for photos/PDFs without bloating database rows.
Authentication	College identity integration where available; otherwise secure email/ID authentication	Reduces duplicate identities and supports institutional control.
Background Jobs	Redis + worker queue	Asynchronous notifications, report generation and cleanup.
Deployment	Managed PostgreSQL + containerized API + managed object storage	Low operational burden for a college-scale deployment.

4. Data Architecture

PostgreSQL should be the system of record for transactional data. Foreign keys, unique constraints, check constraints and transactions should enforce invariants at the database boundary in addition to application-level validation.

Core entities

- User: Identity, authentication status and account metadata.
- Role: Permission grouping such as Volunteer, Leader, Coordinator and Administrator.
- VolunteerProfile: Academic and NSS-specific volunteer information.
- NSSUnit: College NSS unit/group and assigned coordinators.
- UnitMembership: Time-aware relationship between volunteer and unit.
- Event: NSS activity/event definition and lifecycle.
- EventRegistration: Volunteer registration and eligibility status.
- AttendanceSession: Controlled attendance window for an event.
- AttendanceRecord: Verified volunteer attendance with source and timestamps.
- ServiceHourLedger: Auditable calculated/approved service-hour entries.
- ActivityReport: Post-event report and outcomes.
- Document: Metadata for files stored outside the database.
- Announcement: Published notice and audience targeting.
- Notification: Individual delivery record for a user.
- Achievement: Approved award/recognition/leadership record.
- Certificate: Certificate metadata and generated artifact reference.
- AuditLog: Security-relevant and administrative change history.

Simplified relationship model

```
User 1■■■1 VolunteerProfile
VolunteerProfile N■■■N NSSUnit (via UnitMembership)
NSSUnit 1■■■N Event
Event 1■■■N EventRegistration N■■■1 VolunteerProfile
Event 1■■■1 AttendanceSession 1■■■N AttendanceRecord
AttendanceRecord 1■■■N ServiceHourLedger
Event 1■■■0..1 ActivityReport
ActivityReport 1■■■N Document
User 1■■■N Notification
User 1■■■N AuditLog
```

Important data rules

- A volunteer should have at most one active membership in the same NSS unit at a given time.
- An attendance record must reference an existing event and eligible volunteer.
- Only verified attendance contributes to official service-hour totals.
- Corrections to attendance should create a new audit event rather than silently overwriting history.
- Files should be referenced by opaque object keys, not stored directly in normal relational rows.
- Soft deletion should be preferred for records that may be needed for historical reporting.

5. API Architecture

Expose versioned REST endpoints over HTTPS. Use resource-oriented URLs, consistent error envelopes, pagination, filtering, request validation and authorization guards.

Representative API surface

Area	Examples
Auth	POST /api/v1/auth/login, POST /api/v1/auth/logout, POST /api/v1/auth/refresh
Volunteers	GET /api/v1/volunteers, GET /api/v1/volunteers/{id}, PATCH /api/v1/volunteers/{id}
Events	POST /api/v1/events, GET /api/v1/events, PATCH /api/v1/events/{id}
Registration	POST /api/v1/events/{id}/registrations, DELETE /api/v1/events/{id}/registrations/{userId}
Attendance	POST /api/v1/events/{id}/attendance/session, POST /api/v1/attendance/check-in, PATCH /api/v1/attendance/{id}
Reports	POST /api/v1/events/{id}/report, GET /api/v1/reports
Documents	POST /api/v1/documents/upload-url, GET /api/v1/documents/{id}
Announcements	POST /api/v1/announcements, GET /api/v1/announcements
Analytics	GET /api/v1/dashboard, GET /api/v1/reports/attendance

Standard error contract

```
{ "error": { "code": "ATTENDANCE_NOT_ELIGIBLE", "message": "Volunteer is not eligible for this attendance session.", "requestId": "server-generated-id" } }
```

Use stable machine-readable error codes for frontend behavior, while keeping messages safe and understandable. Never expose stack traces, SQL details, secrets or internal infrastructure information.

6. Security Architecture

Security must be designed into every layer rather than added after implementation.

Identity and authorization

- Use short-lived access tokens and securely managed refresh/session mechanisms.
- Hash passwords with a modern password hashing algorithm when passwords are used; never store plaintext credentials.
- Apply role-based access control and resource-level authorization. A coordinator for Unit A must not automatically gain access to Unit B.
- Use server-side authorization even if the UI hides unavailable actions.
- Require re-authentication or elevated authorization for highly privileged operations when appropriate.

Input and API security

- Validate all request bodies, query parameters, path parameters and uploaded-file metadata.
- Use parameterized queries/ORM APIs to prevent SQL injection.
- Apply rate limits to authentication, attendance check-in and other abuse-prone endpoints.
- Use CSRF protections where cookie-based authentication is used.
- Configure secure headers, TLS, CORS allowlists and strict content-type handling.

File security

- Allow only explicitly approved MIME types and size limits.
- Generate server-side object keys; never use user-supplied paths as storage paths.
- Store private files in private buckets and issue short-lived signed download URLs.
- Scan uploaded files for malware where institutional policy and infrastructure support it.

Audit and privacy

- Audit changes to attendance, volunteer status, roles, events, reports and privileged configuration.
- Minimize personal data collection and define retention periods with the college.
- Do not expose volunteer contact information to users who do not need it.
- Backups must be encrypted and access-controlled.

7. Attendance & Service-Hour Workflow

Attendance is a high-integrity domain because it affects official participation records. The workflow should therefore be explicit and auditable.

1. Coordinator creates an event and defines eligible units/volunteers.
2. Volunteer registers if registration is required.
3. Coordinator opens an attendance session for a bounded time window.
4. Volunteer checks in using the approved mechanism, such as a rotating QR token or coordinator-controlled verification.

5. Backend validates event status, eligibility, session validity, duplicate check-in and relevant anti-abuse rules.
6. Attendance remains pending/verified according to the configured workflow.
7. Only verified attendance contributes to the service-hour ledger.
8. Any later correction creates an auditable change record and recalculates affected totals.

QR attendance recommendation

A static QR code is insufficient because it can be photographed and reused. If QR attendance is implemented, use short-lived, server-generated tokens bound to the event/session and validate them server-side. For stronger integrity, combine the token with authenticated identity and coordinator-controlled session timing. Avoid collecting unnecessary location data merely to improve attendance confidence.

8. Frontend / UX Architecture

The frontend should use role-aware navigation and responsive layouts so the same API can support desktop browsers and future mobile clients.

Volunteer screens

- Dashboard: upcoming events, recent participation, verified service hours and announcements.
- Events: browse, filter, view details and register.
- Attendance: current/past status and attendance history.
- Profile: personal and academic information with editable fields controlled by policy.
- Achievements: approved recognitions and certificates.

Coordinator/Admin screens

- Dashboard: participation statistics, upcoming events and pending approvals.
- Volunteer management: search, filters, unit assignment and status.
- Event management: lifecycle, registrations, attendance and reports.
- Reports: exportable attendance, service-hour and activity summaries.
- Administration: roles, units, configuration and audit review.

UX principles

- Show clear status labels such as Draft, Published, Open, Closed, Completed and Archived.
- Never rely on color alone to communicate status.
- Make destructive actions explicit and require confirmation.
- Preserve unsaved form data when safe and provide actionable validation errors.

9. Deployment & Infrastructure

A managed deployment is preferable for a college-scale system. Keep the initial infrastructure simple and observable.

Production topology

Internet
■

[Reverse Proxy / Load Balancer]


```
[ API Container(s) ] ■■■ [ Worker Container ]
```

111

XX

▼ ▼

[PostgreSQL] [Redis]

L

▼

[Encrypted Backups]

```
[Private Object Storage] ←■■ API signed upload/download
```

- Unit tests: service-hour calculation, eligibility rules, event state transitions, permission policies and validation.
- Integration tests: database transactions, attendance workflows, registration constraints and audit creation.
- API tests: authentication, authorization, validation, pagination, error contracts and idempotency.
- End-to-end tests: volunteer registration → event registration → attendance → verified hours → report.
- Security tests: broken access control, IDOR, injection, rate-limit bypass, unsafe file upload and session handling.
- Migration tests: verify forward migrations and backup restoration on a representative database.

Boundary cases that must be tested

- Duplicate registration and duplicate attendance.
- Attendance after session expiry.
- Volunteer removed from a unit after registration.
- Event cancellation after registrations exist.
- Zero-duration or invalid-duration events.
- Concurrent check-in requests from the same volunteer.
- Unauthorized access to another unit's records.
- Oversized, invalid or malicious document uploads.

12. Observability & Auditability

- Use structured logs containing timestamp, request ID, route, status code, duration and actor ID where appropriate.
- Track metrics for request rate, error rate, latency, database connections, job failures and notification delivery.
- Alert on authentication abuse, repeated server errors, database saturation and failed background jobs.
- Audit privileged actions separately from ordinary debug/application logs.
- Avoid logging passwords, tokens, session secrets or unnecessary personal information.

13. Backup, Recovery & Business Continuity

- Automated encrypted PostgreSQL backups with a documented retention policy.
- Point-in-time recovery should be enabled if supported by the selected managed database.
- Object storage should use versioning or protected retention for important reports/documents where appropriate.
- Perform scheduled restoration drills; a backup that has never been restored is not a verified recovery mechanism.
- Document recovery procedures, responsible personnel, expected recovery time and acceptable data-loss window.

14. Implementation Roadmap

Phase 1 — Foundation

- Project setup, database schema, authentication, roles and unit management.

Phase 2 — Core NSS Operations

- Volunteer profiles, events, registrations and attendance.

Phase 3 — Reporting & Communication

- Service hours, activity reports, documents, announcements and notifications.

Phase 4 — Administration & Quality

- Audit logs, exports, dashboards, security hardening, backup/recovery and production monitoring.

Phase 5 — Optional Extensions

- Native mobile app, college SSO, advanced analytics, certificate automation and multi-college support.

15. Key Risks & Mitigations

Risk	Impact	Mitigation
False attendance	High	Short-lived tokens, authenticated identity, controlled sessions and audit logs.
Over-privileged coordinator	High	Unit-scoped RBAC and server-side authorization.
Data loss	High	Encrypted automated backups plus restoration drills.
File abuse	Medium/High	MIME/size validation, private storage, signed URLs and malware scanning where available.
Manual data inconsistency	Medium	Database constraints, transactions, idempotency and authoritative service-hour ledger.
Infrastructure complexity	Medium	Use modular monolith and managed services before considering microservices.

16. Final Architecture Recommendation

For a single college NSS deployment, the recommended architecture is a secure modular monolith using React/TypeScript on the client, a TypeScript/NestJS REST API, PostgreSQL as the transactional source of truth, private S3-compatible object storage for documents, and a small background-worker layer for asynchronous tasks. This design is simpler to operate than microservices, supports strong transactional consistency for attendance and service hours, and leaves clear boundaries for future extraction if the system later expands to multiple colleges.

The architecture should be implemented from the requirements questionnaire and validated with NSS programme officers before database migrations or production deployment are finalized. Institutional policies should determine data retention, certificate validity, official service-hour rules, identity integration and access to personal information.

Trade-offs

Memory: PostgreSQL + API + optional Redis/worker footprint is moderate; object storage prevents large media from consuming database memory/storage resources.

Latency: A modular monolith minimizes network hops; indexed queries and asynchronous jobs keep interactive operations responsive.

Operational Overhead: Managed database/storage plus one primary API deployment keeps administration low, while auditability and backups add necessary governance overhead.